

MYC Command-router

Author: DK1RI

Version V02.00.01 20260618

This paper is published in <https://github.com/dk1ri> as well

Definitions and formats

see <https://dk1ri.de/myc/Definitions.txt> or <https://dk1ri.de/myc/Definitions.pdf>

Introduction

The actual version of the program is basically working but may have some errors may crash.

The intention of the usage of the command-router with integrated logic-device (CR) is to connect one or more devices (FUs) to a single control program (SK), which may be a terminal, a Web-browser (with web-server) or something else. The CR can be used simply without understanding details of the MYC system.

Please read “Installation and usage” below.

For better understanding please read “Details” below.

Installation and usage (Windows)

Download <https://dk1ri.de/myc/commandrouter.zip> (Version V03.02 or later) to any directory and unzip this.

Click the file _INSTALL.bat

This will copy the data to the ProgramData directory.

Additionally you must install python. You will find details in the internet.

If you want to use the Webserver, read <https://dk1ri.de/myc/webserver.pdf> or

<https://dk1ri.de/myc/webserver.txt> and install the program as described. The default configuration will allow the communication between CR and webserver via filetransfer.

The CR must have information about the FUs to connect. You must find out the connection – the COM port for example – and write this to the commandrouter_config/connection_of_devices file. Details can be found within this file.

You can connect FUs via COM Port (USB / RS232) or FILE interface now. Telnet may work, but is not tested.

This must be provided manually now (may be done automatically later).

At start the CR will find the possible SK interfaces, the connected devices and copy the announcements line by line from the devices unless the CR has an announcelist of this devicetype already.

You can copy the commandrouter.bat to the desktop. This will simplify the calling of the program. Nothing else must be done for installation.

After starting the CR you can control the FUs using a terminal or other programs. Command input is binary. You must understand the MYC command language. Using the web-server and a browser, this is not necessary.

Some Details

For more details about the MYC system please read dk1ri.de/emyc.html or <https://dk1ri.de/dmyc.html> (german)

You can control the CR via terminal, USB / RS232 or file-interface as well. Telnet (port23 only) may work, but is not tested.

USB/ RS232 require a virtual COM-port bridge like com0com or similar (if the inut is on the same computer) and a program to input the data and serial output like hterm or similar. If you installed com0com the program is active always. You must connect the 2nd comport by hterm or similar whenever you use the CR. **Otherwise your CR will hang after sending something to the SK!!!**

Input from the terminal, telnet and the file interfaces are given as 2 byte figures (0-9,A-F) per byte. These inputs are converted to bytes. The serial interface uses bytes directly, but the input via hterm can be in other formats as well.

Output to terminal uses bytes, so that text is readable. Non printable characters are given as 0xxx.

Serial output uses bytes as well. File transfer and telnet use 2 byte hex values as the output.

Inputs / outputs of SK and FUs use the same format.

Usually the CR read the announcements from the FU. For some interfaces as some radio connections the transmission may be too long or long transmissions are not recommended at all.

You can copy the announcement-file directly to the devices directory with the name of the devicetype extracted from the basic command.. See https://dk1ri.de/myc/Reserves_token.txt or https://dk1ri.de/myc/Reserves_token.pdf. Older FU have announcement file like "announcement.bas". You mus rename them!

The announcement file may have comment lines starting with "#" or "". and line may have the format as used with BASCOM firmware: DATA "<announceline>".

The program is written in python using pycharm under windows.

Since Version V03,00.00 the LD is integrated in the CR. Both parts of the program work widely independent. The LD uses some data of the CR. Data exchange between the two parts is done by some variables. For details of the LD see <https://dk1ri.de/myc/logicdevice.txt> or <https://dk1ri.de/myc/logicdevice.pdf>.

The MYC protocol is mainly an OSI layer 7 protocol (application layer). It describes (semantic) functions and addressing as well. So the commands and answers can be seen as messages.

The basic function of the CR is to route incoming commands to the FUs handling these commands. It require correct data because the commands have no header or trailer to show, when a command starts or end. The CR do some checks but wrong data may result in unwanted commands.

There is a short timeout, so, if the command input rate is not too high, the CR start again.

At start some lists are generated; they speed up the handling of commands and answers. With a standard Windows laptop the CR can handle 100 to 500 commands per second but depending on the IO to SK and FU it may be less.

Some limitations

The code is tested on windows and on a raspberry (not the actual version)

The number of elements for "xa" / "xb" command is limited to 255 elements.

The stringlength is limited to 255 bytes.

Length of announcelines is limited to 255 bytes

Main missing topics

see still missing in the program directory

Programming principles

This part describes my implementation of the command-router (CR) and my principles when writing the program. I hope, that the documentation within program is sufficient.

This was my first python program, so enhancements are necessary.

Coding is done quite basic, so object oriented programming is rarely used.

Nevertheless for understanding the program you must understand the MYC principles and the protocol details. For basic understanding at least see <https://dk1ri.de/myc/Description.txt> or <https://dk1ri.de/myc/Description.pdf> and <https://dk1ri.de/myc/commands.txt> or <https://dk1ri.de/myc/commands.pdf>

It may be, that a standard messaging software can be used. When I wrote the CR I did not found a suitable one. I did not want to add an addition protocol stack, which comes with those software.

Nevertheless there may be other solutions for the CR.

The program uses polling. After initialization following functions are polled:

time-dependent task

CR read input SK (keyboard, USB, FILE, network)

CR check SK-input-buffer

CR send to LD

LD execute rule checking

CR check input-buffer from LD

CR transmit command to device

CR read devices (serial, FILE)

CR check device-inputbuffer (answer or info)

CR send answer / info to LD

LD update status if applicable

LD send answer to CR

CR send answers to SK-buffer

output to SK

Because the CR must analyze any incoming byte, many lists and arrays are created during initialization for speed up. They are defined as `v_XXX.py` global variables. I am not sure, whether the used data structure is the most effective (in terms of speed). So enhancements may be necessary.

Sometimes the program lines are quite long and subarrays are hard to read. So I sometimes introduces variables like `temp` with a “lifetime” over a few lines. This enhances the readability, but may reduce speed. I also used those variables, when they are needed a few times, because a usage of a variable may be faster than an element of a subarray.

There are 3 input parser subprograms:

- `command_handling`: analyzing the SK-buffer for commands)
- `ld_command_handling`: analyzing the LD buffer
- `device_handling`: analyzing the device buffer

These subprograms use the same parser `data_handling`

Speed (no LD)

Using the random test, the number of input events can be seen.

Surprisingly the Raspberry with ArmV71 was faster than a Windows10 Core i7-8550: 1946 / 1498 events / second measured with an older CR version.

Testing

If the variable `v_cr_params.test_mode` is set to 1, log entries and some other messages are sent to the terminal.

There are two test devices, see https://dk1ri.de/myc/cr_test.pdf

Implementation of the Hardware

The CR should be implemented on a Raspberry PI.

May be that a web server for the SK can also run on the same machine..
not yet tested.

Hierarchical MYC System

In a hierarchical system the CR resolves the announcements of lower level CR but otherwise keep the sequence. So only the actual CR delivers the announce-list with the basic announcement at start and the reserved tokens and the I-line of the CR at the end (tabs for clarification only):

<code><0>;c;...</code>	#CR of actual level
<code><c><m>...</code>	#normal device of actual level
...	
<code><c>;c;...</code>	#lower level CR
<code><c><m>...</code>	#normal device of lower level
...	
<code><c><c>...</code>	# other lower level CR
...	
...	
<code><c><c>..</code>	# next level

Identical device-types with identical individualization are not allowed within a CR but for CR in different levels.

Error handling

The routing kernel of the CR mainly uses byte-array. Any (hex)character is allowed as input.

The CR do some checks on the inputs:

- for valid command-token
- for positional parameters and element number lower than maximum
- for correct length of string

`<data>` fields are not checked (with few exceptions), especially restriction of `<ty>` given in `<des>`.

Error messages are sent to log-file only.

If an error is detected, received characters are ignored and the next byte is seen as the start of the next command / answer.

Multi-user

Some interfaces will support multi-user access. This is not supported now.

“Q” rules are not supported now

Copyright

Dieses Dokument darf unverändert kopiert werden.

Die Ideen in diesem Dokument unterliegen der GPL (Gnu Public Licence,V2) soweit keine früheren, anderen Rechte betroffen sind.

Die Verwendung der Unterlagen erfolgt auf eigene Gefahr; es wird keinerlei Garantie übernommen.
This document can be copied without changes.
The ideas of this document can be used under GPL (Gnu Public License, V2) as long as no earlier
other rights are affected.
The usage of this document is on own risk, there is no warranty.